

Лекция 2

Основы объектно-ориентированного программирования

1 Основные принципы ООП

Абстракция — это придание объекту характеристик, которые отличают его от всех объектов, четко определяя его концептуальные границы.

Инкапсуляция — свойство, позволяющее пользователю не задумываться о сложности реализации используемого программного компонента (что у него внутри?), а взаимодействовать с ним посредством предоставляемого интерфейса (публичных методов и членов), а также объединить и защитить жизненно важные для компонента данные.

Наследование — свойство, позволяющий описать новый класс на основе уже существующего (родительского), при этом свойства и функциональность родительского класса заимствуются новым классом.

Полиморфизм — возможность объектов с одинаковой спецификацией иметь различную реализацию. Кратко смысл полиморфизма можно выразить фразой: «Один интерфейс, множество реализаций».

2 Понятия класса и объекта

Класс - это абстракция, описывающая свойства и методы ещё не существующих объектов (пользовательский тип данных).

Поле класса — это данные, содержащиеся внутри класса.

Методы класса — это функции, входящие в состав класса.

```
// объявление классов в C++
class /*имя класса*/
{
    private:
    /* список свойств и методов для использования внутри класса */
    public:
    /* список методов доступных другим функциям и объектам программы */
    protected:
    /*список средств, доступных при наследовании*/
};
```

```

1 class People
2 {
3     string FIO;
4     int AGE;
5
6     public:
7
8     void Set_P(string fio, int age)
9     {
10         FIO = fio;
11         AGE = age;
12     }
13
14     void Show_P()
15     {
16         cout << "ФИО: " << FIO << endl;
17         cout << "Возраст: " << AGE << endl;
18     }
19
20 };

```

```

1 class People
2 {
3     // в классе все поля и методы закрытые
4
5     string FIO;
6     int AGE;
7
8     void Set_P(string fio, int age)
9     {
10         FIO = fio;
11         AGE = age;
12     }
13
14     void Show_P()
15     {
16         cout << "ФИО: " << FIO << endl;
17         cout << "Возраст: " << AGE << endl;
18     }
19
20 };

```

```

1 class People
2 {
3     // в классе все поля и методы открытые
4     public:
5
6     string FIO;
7     int AGE;
8
9     void Set_P(string fio, int age)
10    {
11        FIO = fio;
12        AGE = age;
13    }
14
15    void Show_P()
16    {
17        cout << "ФИО: " << FIO << endl;
18        cout << "Возраст: " << AGE << endl;
19    }
20
21 };

```

```

1 People P;
2
3 People M[20], Anna;
4
5 People *Mas;
6

```

3 Определение методов в классе и вне класса

Доступ к методам класса возможен
через конкретный объект этого класса
при помощи операции точки

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // определение класса
6 class People
7 {
8     string FIO;
9     int AGE;
10
11     public:
12
13     // установка данных
14     void setPeople(string fio, int age)
15     {
16         FIO = fio;
17         AGE = age;
18     }
19
20     // вывод данных
21     void showPeople()
22     {
23         cout << "ФИО: " << FIO << endl;
24         cout << "Возраст: " << AGE << endl;
25     }
26 };
27
28 int main()
29 {
30     People Ann;
31     Ann.setPeople("Иванова Анна Ивановна", 20);
32     Ann.showPeople();
33 }
```

ФИО: Иванова Анна Ивановна
Возраст: 20

Определение метода вне класса

Тип_метода Имя_класса :: Имя_метода (Список_параметров)

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // определение класса
6  class People
7  {
8      string FIO;
9      int AGE;
10
11     public:
12
13     // установка данных (прототип метода)
14     void setPeople(string fio, int age);
15
16     // вывод данных
17     void showPeople()
18     {
19         cout << "ФИО: " << FIO << endl;
20         cout << "Возраст: " << AGE << endl;
21     }
22 };
23
24 // определение метода вне класса
25 void People::setPeople(string fio, int age)
26 {
27     FIO = fio;
28     AGE = age;
29 }
```

Символ (::) является знаком *операции глобального разрешения*.

```
30
31 int main()
32 {
33     People Ann;
34     Ann.setPeople("Иванова Анна Ивановна", 20);
35     Ann.showPeople();
36 }
```

```
ФИО: Иванова Анна Ивановна
Возраст: 20
```

ПРИНЯТО:

- методы SET – изменение данных;
- методы GET – получение данных.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // определение класса
6  class People
7  {
8      string FIO;
9      int AGE;
10 public:
11
12     People() : FIO ("Петров Иван Петрович"), AGE (30) {}
13
14     int Get_AGE () { return AGE; }
15     void Set_AGE (const int &age) { AGE = age; }
16
17     string Get_FIO () { return FIO; }
18     void Set_FIO (const string &fio) { FIO = fio; }
19
20     void showPeople()
21     {
22         cout << endl << "ФИО: " << FIO << endl;
23         cout << "Возраст: " << AGE << endl;
24     }
25 };
```

ФИО: Петров Иван Петрович

Возраст: 30

ФИО: Иванов Иван Александрович

Возраст: 30

ФИО: Иванов Иван Александрович

Возраст: 10

Иванов Иван Александрович

```
26
27 int main()
28 {
29     People Ivan;
30     Ivan.showPeople();
31
32     Ivan.Set_FIO("Иванов Иван Александрович");
33     Ivan.showPeople();
34
35     Ivan.Set_AGE(10);
36     Ivan.showPeople();
37
38     if (Ivan.Get_AGE() == 10)
39     {
40         cout << endl << Ivan.Get_FIO();
41     }
42 }
```

3 Конструкторы и деструкторы

Конструктор — специальный метод для инициализации полей объекта класса, являющийся членом класса и имеющий то же имя:

```
Имя_класса( Список_аргументов ): поле1 (значение), поле2 (значение)  
    { /*например, пустое тело*/ }
```

Нельзя вызывать функцию конструктора в явном виде.
Во время создания объекта, при выделении памяти,
вызывается конструктор.

Для освобождения памяти из-под объекта, т. е. при прекращении действия объекта, выполняется **деструктор**, имеющий имя:

~Имя_класса


```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // определение класса
6  class People
7  {
8      string FIO;
9      int AGE;
10
11     public:
12
13     // конструктор без параметров
14     People() : FIO ("Петров Иван Петрович"), AGE (30)
15     { }
16
17     // конструктор с параметрами
18     People(string fio, int age) : FIO (fio), AGE(age)
19     {
20         cout << "Сработал конструктор с параметрами" << endl;
21     }
22
23     // конструктор
24     People(string fio)
25     {
26         FIO = fio; AGE = 40;
27     }
28
29     void showPeople()
30     {
31         cout << "ФИО: " << FIO << endl;
32         cout << "Возраст: " << AGE << endl;
33     }
34 };

```

```

35
36 int main()
37 {
38     People Ivan;
39     Ivan.showPeople();
40
41     People Ann("Иванова Анна Ивановна", 20);
42     Ann.showPeople();
43
44     People Alex("Сидоров Александр Александрович");
45     Alex.showPeople();
46 }

```

```

ФИО: Петров Иван Петрович
Возраст: 30
Сработал конструктор с параметрами
ФИО: Иванова Анна Ивановна
Возраст: 20
ФИО: Сидоров Александр Александрович
Возраст: 40

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // определение класса
6  class People
7  {
8      string FIO;
9      int AGE;
10
11     public:
12
13     // конструктор без параметров
14     People() : FIO ("Петров Иван Петрович"), AGE (30)
15     { }
16
17     // деструктор
18     ~People()
19     {
20         cout << "Сработал деструктор";
21     }
22
23     void showPeople()
24     {
25         cout << "ФИО: " << FIO << endl;
26         cout << "Возраст: " << AGE << endl;
27     }
28 };
29
30 int main()
31 {
32     People Ivan;
33     Ivan.showPeople();
34 }

```

ФИО: Петров Иван Петрович
Возраст: 30
Сработал деструктор

Вызов деструктора:

```

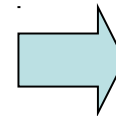
25 int main()
26 {
27     People * Ivan1 = new People();
28     Ivan1->showPeople();
29     // принудительный вызов деструктора
30     Ivan1->~People();
31 }

```

4 Объекты как аргументы методов и доступ к их членам

```
1  #include <iostream>
2  using namespace std;
3
4  class OBJ
5  {
6      int i;
7      public:
8          void set_i(int x) { i=x; }
9          void out_i() { cout<<i<<endl; }
10 };
11
12 // прототипы
13 void Fun1(OBJ x);
14 void Fun2(OBJ *x);
15
16 int main(void)
17 {
18     OBJ A;
19     A.set_i(10);
20     A.out_i();
21
22     Fun1(A);
23     A.out_i();
24
25     Fun2(&A);
26     A.out_i();
27 }
```

```
28
29 void Fun1(OBJ x)
30 {
31     cout<<"Начало Fun1"<<endl;
32     x.out_i();      // вывод x
33     x.set_i(100) ;// изменение x
34     x.out_i();      // вывод x=100
35     cout<<"Конец Fun1"<<endl;
36 }
37
38 void Fun2(OBJ *x)
39 {
40     cout<<"Начало Fun2"<<endl;
41     x->out_i();      // вывод x
42     x->set_i(200) ;// изменение x
43     x->out_i();      // вывод x=200
44     cout<<"Конец Fun2"<<endl;
45 }
```



```
10
Начало Fun1
10
100
Конец Fun1
10
Начало Fun2
10
200
Конец Fun2
200
```

5 Конструктор копирования

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // определение класса
6 class People
7 {
8     string FIO;
9     int AGE;
10 public:
11     // конструктор
12     People(string fio, int age) : FIO (fio), AGE (age) { }
13
14     void showPeople()
15     {
16         cout << "ФИО: " << FIO << endl;
17         cout << "Возраст: " << AGE << endl;
18     }
19 };
20
21 int main()
22 {
23     People Ivan("Иванов И.И.", 15);
24     Ivan.showPeople();
25
26     People Alex(Ivan);
27     Alex.showPeople();
28
29     People Petr = Ivan;
30     Petr.showPeople();
31 }
```

Имя_класса (const Имя_класса &)

...

Имя_класса obj1 (...);

Имя_класса obj2 (obj1);

Имя_класса obj3 = obj1.

```
ФИО: Иванов И.И.
Возраст: 15
ФИО: Иванов И.И.
Возраст: 15
ФИО: Иванов И.И.
Возраст: 15
```

Код конструктора копирования?

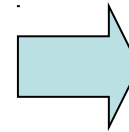
6 Статические поля и методы класса

```
1  #include <iostream>
2  using namespace std;
3
4  class MyClass
5  {
6      int N;
7      static int S; // статическое поле
8  public:
9      void Set_N(int n) { N=n; }
10     void Show() {cout << "N = " << N << ", S = " << S << endl;}
11     // статические методы
12     static int Set_S(int s) { S = s; }
13     static void Show_S() {cout << "S = " << S << endl;}
14 };
15
16 // инициализация статического свойства
17 int MyClass::S=30;
18
19 int main()
20 {
21     MyClass::Show_S();
22
23     MyClass Obj1;
24     Obj1.Set_N(1);
25     Obj1.Show();
26
27     MyClass Obj2;
28     Obj2.Set_N(2);
29     Obj2.Show();
30
31     MyClass::Set_S(40);
32     Obj1.Show_S(); // не грамотно
33     Obj1.Show(); Obj2.Show();
34 }
```

```
S = 30
N = 1, S = 30
N = 2, S = 30
S = 40
N = 1, S = 40
N = 2, S = 40
```

7 Массив объектов

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 // определение класса
5 class People
6 {
7     string FIO;
8     int AGE;
9 public:
10     People() : FIO ("Петров Иван Петрович"), AGE (30) {}
11     People(string fio, int age) : FIO (fio), AGE (age) { }
12     void showPeople()
13     {
14         cout << "ФИО: " << FIO << endl;
15         cout << "Возраст: " << AGE << endl;
16     }
17 };
18 int main()
19 {
20     People Mas[3];
21     for(int i=0;i<3;i++)
22     {
23         cout << endl << i << endl;
24         Mas[i].showPeople();
25     }
26     People P0("Сидорова Анна Ивановна", 20);
27     Mas[0] = P0;
28     Mas[2] = People("Иванов Иван Иванович", 10);
29     for(int i=0;i<3;i++)
30     {
31         cout << endl << i << endl;
32         Mas[i].showPeople();
33     }
34 }
```



```
0
ФИО: Петров Иван Петрович
Возраст: 30

1
ФИО: Петров Иван Петрович
Возраст: 30

2
ФИО: Петров Иван Петрович
Возраст: 30

0
ФИО: Сидорова Анна Ивановна
Возраст: 20

1
ФИО: Петров Иван Петрович
Возраст: 30

2
ФИО: Иванов Иван Иванович
Возраст: 10
```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // определение класса
6  class People
7  {
8      string FIO;
9      int AGE;
10     public:
11
12     People() : FIO ("Петров Иван Петрович"), AGE (30) {}
13     People(string fio, int age) : FIO (fio), AGE (age) { }
14
15     void showPeople()
16     {
17         cout << "ФИО: " << FIO << endl;
18         cout << "Возраст: " << AGE << endl;
19     }
20
21     int Get_AGE () { return AGE; }
22     void Set_AGE (const int &age) { AGE = age; }
23 };

```

```

24
25 int main()
26 {
27     People Mas[3];
28     Mas[0].Set_AGE(10);
29     Mas[2].Set_AGE(15);
30
31     for (int i = 0; i<3; i++)
32     {
33         cout << endl << i << endl;
34         Mas[i].showPeople();
35     }
36
37     // поиск людей моложе 20 лет
38     cout << endl << "МОЛОЖЕ 20 ЛЕТ:" << endl;
39     for (int i = 0; i<3; i++)
40     {
41         if (Mas[i].Get_AGE() < 20)
42         {
43             cout << endl << i << endl;
44             Mas[i].showPeople();
45         }
46     }
47
48 }

```

```

0
ФИО: Петров Иван Петрович
Возраст: 10

1
ФИО: Петров Иван Петрович
Возраст: 30

2
ФИО: Петров Иван Петрович
Возраст: 15

```

```

МОЛОЖЕ 20 ЛЕТ:

0
ФИО: Петров Иван Петрович
Возраст: 10

2
ФИО: Петров Иван Петрович
Возраст: 15

```